



– Algorithmique – TP N°5 – PROCEDURES & FONCTIONS –

I– Introduction : Les **procédures et fonctions** sont utilisées pour **simplifier** l'écriture des programmes.

Le programme global est ainsi **décomposé** en plusieurs parties faciles à écrire, à comprendre et à faire évoluer à l'inverse d'un programme de grande taille écrit en un seul bloc.

Le programme (**principal**) décrit alors la méthode générale de résolution du problème traité et les procédures et fonctions traitent les détails.

Cette décomposition permet aussi de **réutiliser** les procédures et fonctions au lieu de les réécrire à chaque fois car souvent les mêmes traitements sont effectués plusieurs fois dans le même programme.

Résumé : ① En langage C, tous les traitements sont effectués par des fonctions. ② Les procédures sont un cas particulier de fonctions. ③ Le programme principal est la fonction **int main ()**.

II– Les fonctions

✓ Une fonction retourne un seul résultat avec l'instruction **return**. Elle est définie selon la syntaxe :

```
typeResultat nomFonction(TypeParametre1 parametre1, ... , TypeParametreN parametreN)
{
    /* Déclarations de variables locales */
    /* Liste des instructions*/
    return resultat ;    }
```

typeResultat : est l'un des types ENTIER (char, short, int, long, ...) , REEL (float, double, ...), CARACTERE (char) ou ENREGISTREMENT (struct).

Exemple : Une fonction qui calcule le **maximum** de deux **entiers n1** et **n2** peut être écrite comme suit :

```
int maximum(int n1, int n2) { int max ; // max est une variable locale
    if(n1 > n2) max = n1 ;
    else max = n2 ;
    return max ; }
```

✓ L'instruction **return** permet de retourner le résultat et terminer la fonction (ou la procédure). Si l'instruction **return** est rencontrée, toutes les instructions qui la suivent ne seront pas exécutées.

Exemple : La fonction **maximum** précédente peut être réécrite comme suit :

```
int maximum(int n1, int n2){ if(n1 > n2) return n1 ;
    return n2 ; }
```

✓ Une fonction est, généralement, utilisée (appelée) dans une expression arithmétique ou logique. Elle peut aussi servir comme paramètre (passé par valeur) à une autre fonction ou procédure.

Exemples d'appels : ① `int n = n3 + maximum( n1, n2) ;` ② Pour trouver le maximum de trois entiers, on peut écrire : `int max = maximum( maximum(n1, n2), n3) ;` // Le 1^{er} paramètre est un appel de fonction.

III – Les procédures

✓ Une procédure est une fonction dont le type de retour est **void** (vide). Il signifie que la procédure ne retourne aucune valeur. **Syntaxe de déclaration :** `void nomProcédure( Liste_des_paramètres )`

- ✓ La procédure est utilisée (appelée) comme une instruction. Elle ne peut pas être utilisée dans une expression arithmétique ou logique, ni passée en paramètre à une autre procédure ou fonction.
- ✓ Dans une procédure, l'utilisation de l'instruction **return** reste possible, mais elle n'est pas suivie d'un résultat (**return ;**).

Remarque 1 : Les instructions **scanf** et **printf** sont, en réalité, deux fonctions. ① **scanf** : En plus des valeurs lues à partir du clavier, elle retourne le nombre de variables correctement lues à partir du clavier.
② **printf** : En plus de l'affichage sur écran, cette fonction retourne le nombre de caractères affichés.

Remarque 2 : Si une fonction est appelée comme une procédure (comme une instruction), la fonction sera exécutée mais la valeur de retour n'est pas exploitée (utilisée). **Exemples** : **scanf** et **printf**.

IV – Passage de paramètres par valeur (par copie) : Dans ce type de passage de paramètres, la valeur du paramètre est copiée dans une variable locale sans toucher la valeur d'origine. C'est cette variable locale qui est utilisée dans la fonction (ou la procédure) appelée. Aucune modification de la variable locale dans la fonction ne modifie la variable passée en paramètre, parce que ces modifications ne s'appliquent qu'à une copie de cette dernière.

Exemple : La fonction **facto** qui calcule la factorielle d'un entier positif **n** peut être écrite comme suit :

```
int facto(int n) { int f = 1 ; for( ; n > 1 ; n--) f *= n ; return f ; }
```

Explication : Malgré que la fonction **facto** modifie le paramètre **n** en le décrémentant (**n--**), il retrouvera sa valeur initiale à la fin de la fonction car cette dernière travaille sur une copie de **n** (**n** est passé par valeur).

- ✓ Un paramètre passé par valeur (par copie) peut recevoir une valeur constante, une variable, une expression ou même un appel à une fonction. **Exemples** : **facto(5)**, **facto(x)**, **facto(n1 + n2)**, **facto(maximum(n1, n2))**.

V – Passage de paramètres par adresse (par référence) : Dans ce type, il n'y a pas de copie de la valeur du paramètre mais la copie de son adresse. La procédure (ou fonction) travaille directement avec la variable passée en paramètre, elle peut ainsi modifier sa valeur. Toute modification du paramètre dans la procédure appelée entraîne la modification de la variable passée en paramètre.

- ✓ Un paramètre passé par adresse à une procédure (**rarement** à une fonction) est précédé par le symbole *****.

Exemple : **int *a** ; La variable **a** est dite un pointeur sur un **int**. (***a**) est la valeur pointée par le pointeur **a**.

Une procédure qui incrémente la valeur d'un **int a** peut être écrite comme suit :

```
void incrementer(int *a) { (*a)++ ; /* ou *a = *a + 1 ; */ }
```

Exemple d'appel de la procédure : **int n = 10 ; incrementer(&n) ; printf("%d", n) ;** // n aura la valeur 11.

A l'appel, il faut utiliser le symbole **&** (lu adresse) comme déjà utilisé dans **scanf**.

VI – Variables locales et variables globales

- ✓ Une **variable globale** est déclarée à l'extérieur du corps de toute fonction, et peut donc être utilisée dans n'importe quelle procédure ou fonction (y compris la fonction **main**). En général, les variables globales sont déclarées au début du programme, immédiatement après les directives **#include** et **#define**.

- ✓ Une **variable locale** ne peut être utilisée que dans la fonction ou le bloc ({ ...bloc... }) où elle est définie.

Remarques : ① Si une variable globale n'est pas initialisée par l'utilisateur, elle est initialisée par défaut à zéro. ② Si une variable locale n'est pas initialisée par l'utilisateur, sa valeur est indéterminée. **Attention !**

VII – LES EXERCICES – LES PROCEDURES ET LES FONCTIONS

Exercice 1 : Tables de multiplication

Q1) Ecrire une procédure *afficherMultiples* qui permet d'afficher la table de multiplication d'un entier n compris entre 1 et 10 ($1 \leq n \leq 10$). En face, la table de multiplication de $n = 7$.

Q2) Ecrire le programme principal (la fonction *int main()*) qui permet de lire un entier n compris entre 1 et 10 et d'afficher sa table de multiplication.

Q3) Modifier le programme principal pour afficher toutes les tables de multiplication de 1 à 10.

7	×	1	=	7
7	×	2	=	14
7	×	3	=	21
7	×	4	=	28
7	×	5	=	35
7	×	6	=	42
7	×	7	=	49
7	×	8	=	56
7	×	9	=	63
7	×	10	=	70

Exercice 2 : Affichage d'étoiles

Pour chaque procédure, écrire un programme C qui permet de la tester.

Q1) Ecrire une procédure *ligne(n)* qui permet d'afficher une ligne de n étoiles '*' (astérisques).

Exemple : ***** est une ligne de 10 étoiles.

Q2) En utilisant la procédure *ligne*, écrire une procédure *rectangle(n, m)* qui permet d'afficher un rectangle d'étoile de m lignes de n étoiles. **Exemple :** Ci-dessous un rectangle de 3 lignes de 20 étoiles.

Q3) En utilisant la procédure *rectangle*, écrire une procédure *carre(n)* qui permet d'afficher un carré d'étoiles de n lignes de n étoiles. Ci-dessous un carré de 4 lignes de 4 étoiles.

Exercice 3 : Ordre croissant

Q1) Pour deux entiers a et b , écrire une procédure *Trier2* qui permute a et b , si nécessaire, pour que l'état de sortie soit $a \leq b$ (a et b seront dans l'ordre croissant).

Q2) En utilisant la procédure *Trier2*, écrire une procédure *Trier3* sur 3 entiers (a , b et c) qui permet d'avoir les trois variables a , b et c dans l'ordre croissant ($a \leq b \leq c$).

Q3) Ecrire un programme C qui permet de lire 3 entiers (x , y , z) puis les afficher dans l'ordre croissant.

Exercice 4 : Maximum/Minimum

Q1) Ecrire une fonction *max2(x, y)* qui retourne le plus grand de deux nombres entiers x et y .

Q2) Utiliser la fonction *max2* pour écrire une deuxième fonction *max3(x, y, z)* qui calcule le plus grand de trois nombres entiers x , y et z .

Q3) Ecrire un programme C qui permet de lire trois entiers a , b , c et d'afficher leur maximum.

***Q4)** Adapter les fonctions précédentes ainsi que le programme principal pour afficher le **minimum** de trois entiers a , b et c .

* Exercice 5 : Nombres Premiers

Un nombre est dit premier s'il n'admet que deux diviseurs : 1 et lui-même.

Q1) Ecrire une fonction **Premier(n)** qui permet de vérifier si un entier **n** est premier ou pas.

Q2) Ecrire une procédure qui permet d'afficher les **m** premiers nombres premiers.

Exemple : Si **m = 8**, la procédure doit afficher : 2 3 5 7 11 13 17 19.

Q3) Ecrire un programme C qui permet de lire un entier **m** et d'afficher les **m** premiers nombres premiers.

Exercice 6 : Fonctions itératives ou récursives ?

Pour chaque fonction, écrire un programme C qui permet de la tester.

Q1) Factorielle : Ecrire une fonction **Facto(n)** qui permet de calculer $n! = 1 * 2 * 3 * \dots * n$

Q2) Puissance : Ecrire une fonction **Puiss(x, n)** qui renvoie $x^n = x * x * \dots * x$ (n fois)

Q3) Factorielle : Ecrire une fonction récursive **FactoRec(n)** qui permet de calculer **n !** en sachant que :

$$n! = n * (n-1)! \text{ et } 0! = 1! = 1$$

Q4) Puissance : Ecrire une fonction récursive **PuissRec(x, n)** qui permet de calculer x^n en utilisant le fait que :

$$x^n = \begin{cases} 1 & \text{Si } n=0 \\ x^{n-1} \times x & \text{Si } n>0 \end{cases}$$

Q5) Puissance : Ecrire une fonction récursive **PuissRec2(x, n)** qui permet de calculer x^n en utilisant le fait

$$\text{que : } x^n = \begin{cases} 1 & \text{Si } n=0 \\ x^{n/2} \times x^{n/2} & \text{Si } n>0 \text{ et } n \text{ pair} \\ x^{(n-1)/2} \times x^{(n-1)/2} \times x & \text{Si } n>0 \text{ et } n \text{ impair} \end{cases}$$

* **Q6)** L'algorithme d'Euclide permettant de calculer le **PGCD** (Plus Grand Commun Diviseur) de deux entiers strictement positifs **A** et **B** tels que **A > B** est défini comme suit :

$$PGCD(A, B) = \begin{cases} PGCD(B, A \bmod B) & \text{Si } B \neq 0 \\ A & \text{Si } B = 0 \end{cases}$$

Ecrire une fonction récursive permettant de déterminer le **PGCD** de **A** et **B**.

* **Q7) Suite de Fibonacci** : Ecrire une fonction récursive **Fibo(n)** permettant de calculer le $n^{ième}$ terme de la

$$\text{suite de } \mathbf{Fibonacci} \text{ définie par : } U_n = \begin{cases} 0 & \text{Si } n = 0 \\ 1 & \text{Si } n = 1 \\ U_{n-1} + U_{n-2} & \text{Si } n \geq 2 \end{cases}$$

Q8) Pour un entier positif **n (n ≥ 0)**, écrire une fonction itérative puis une autre récursive qui permet :

- Calculer le nombre des chiffres de **n**.
- Calculer la somme des chiffres de **n**.
- Calculer le produit (la multiplication) des chiffres de **n**.
- * Vérifier si un chiffre ($0 \leq \text{chiffre} \leq 9$) existe dans le nombre **n**.
- * Calculer le nombre d'occurrences d'un chiffre ($0 \leq \text{chiffre} \leq 9$) dans le nombre **n**.

* : Travail facultatif



Travaux Pratiques: Niveau : 1ere année Semestre: 2 Module: Algorithmique et structure de données II

– Algorithmique – TP N°6 – LES LISTES LINEAIRES CHAINEES –

I- Les enregistrements

I.1- Rappel de cours : Un enregistrement est une structure de données permettant de regrouper dans une seule entité un ensemble de données *de types différents* associées à un même et seul objet. Chaque donnée est appelée un champ. Chaque champ est identifié par *un nom* qui permet d'y accéder directement et *un type*. Le type d'un champ peut être simple (*int, float, char, ...*) ou structuré (*tableau, matrice, enregistrement*).

Exemple : Pour manipuler les informations d'un étudiant, on peut créer un type **Etudiant** comme suit :

Type Etudiant = Enregistrement

Nom : Chaîne
Prenom : Chaîne
Age : Entier
Moyenne : Réel

Fin

Var e1, e2 : Etudiant /* e1 et e2 deux variables de type **Etudiant**, elles ont chacune 4 champs : Nom, Prénom, ... */

I.2- Les enregistrements en langage C

En langage C, le type enregistrement est déclaré avec le mot clé **struct**.

Pour déclarer le type Etudiant en langage C, on écrit :

```
struct Etudiant {  
    char Nom[40] ; /* une chaine de 39 caractères maximum */  
    char Prenom[40] ; /* une chaine de 39 caractères maximum */  
    int Age ;  
    float Moyenne ;  
};  
  
struct Etudiant e1, e2 ; /* déclaration de deux variables de type struct Etudiant */
```

Remarques : ① En général, on utilise le mot clé **typedef** pour renommer le type **struct Etudiant** en **Etudiant** comme suit : **typedef struct Etudiant ;**

La déclaration des deux variables e1 et e2 sera simplifiée comme suit :

```
typedef struct Etudiant ;  
  
Etudiant e1, e2 ; /* Deux variables de type Etudiant */
```

② L'instruction **typedef** peut être placée avant ou après la déclaration du type **struct Etudiant { ... } ;**

I.3- Accès aux champs d'un enregistrement

Pour accéder à un champ, il faut utiliser l'opérateur point (.).

Exemples : - Pour affecter la valeur 20 au champ Age de l'étudiant e1, on écrit : **e1.Age = 20 ;**

- Pour donner la valeur 11.5 au champ Moyenne de l'étudiant e1, on écrit : **e1.Moyenne = 11.5 ;**

II- Les pointeurs en langage C

Rappel : Un pointeur est une variable qui contient l'adresse mémoire d'une autre variable stockée en mémoire.

II.1- Déclaration : Pour déclarer une variable de type pointeur sur un type donné (simple ou structuré), il suffit de précéder sa déclaration par un astérisque (*).

Exemple1 : Pour déclarer un pointeur (appelé ptr) sur un entier (*int*), on peut écrire : `int *ptr ;`

Exemple2 : Pour déclarer un pointeur (appelé p) sur un enregistrement de type Etudiant, on peut écrire :

`Etudiant *p ;`

II.2- Accès à la donnée : Pour accéder à la variable pointée, on écrit par exemple : `*ptr = 20 ;` ou `printf("%d", *ptr) ;`

Remarque : Les pointeurs sont utilisés pour le passage de paramètres par adresse (par variable) dans les procédures (et fonctions), comme déjà vu au **TP N°5 – PROCEDURES & FONCTIONS**.

II.3- Accès aux champs d'un enregistrement pointé

Pour accéder à un champ (le champ Age, par exemple) d'un enregistrement pointé (par la variable p), la notation `(*p).Age` reste valable. Cependant, le langage C, propose une notation plus simple selon la syntaxe suivante : `p->Age`. Pour affecter à l'âge la valeur 20, on écrit alors `p->Age = 20 ;` au lieu de `(*p).Age = 20 ;`

II.4- Initialisation : La constante **NULL** (en majuscule) est utilisée pour indiquer que le pointeur ne pointe vers aucune donnée valide. On écrit alors : `ptr = NULL ;`

II.5- Allocation mémoire : Pour allouer un espace mémoire à la variable pointée, on utilise l'instruction `malloc` de la bibliothèque `stdlib.h`.

Exemple1 : pour réserver l'espace à un entier on écrit : `ptr = malloc( sizeof( int ) ) ;`

Exemple2 : pour réserver l'espace à un enregistrement de type Etudiant pointée par le pointeur p, on écrit :

`p = malloc( sizeof( Etudiant ) ) ;`

II.6- Libération de l'espace mémoire : Pour libérer l'espace mémoire (précédemment alloué par `malloc`) et pointé par le pointeur *ptr*, on utilise l'instruction `free` de la bibliothèque `stdlib.h`.

On écrit alors : `free( ptr ) ;`

III- Les listes en langage C

III.1- Rappel : Une Liste Linéaire Chaînée (LLC) est un ensemble d'éléments (Cellule, Nœud, Maillon) alloués dynamiquement chaînés (reliés) entre eux. Chaque élément est un enregistrement qui contient au moins deux champs : ①- Un champ qui contient l'information (appelé Valeur, **Val**, Info ou Data).

②- Un champ qui est un pointeur sur l'élément suivant (appelé **Suiv** ou Suivant).

III.2- Déclaration : Dans un programme C, le type Liste (d'entiers) peut être déclaré comme suit :

```
typedef struct Element Element;
typedef Element* Liste;
struct Element {
    int val; /* val Contient l'information utile */
    Liste suiv; /* suiv un pointeur vers l'élément suivant */
};
Liste L, P, Q, Ptr ; /* L, P, Q, Ptr : des variables de type Liste ou pointeur sur Element */
```

III.3- Accès aux champs

- ✓ On accède au champ **val** par **L->val**. Exemples : **L->val = 25 ;** ou **printf("%d\n", L->val) ;**
- ✓ On accède au champ **suiv** par **L->suiv**. Exemples : **L->suiv = NULL ;** ou **if (L->suiv != NULL) ...**

III.4- Allocation (création)

L = malloc(sizeof(Element)) ;

- ✓ Si l'opération d'allocation s'est bien déroulée, l'instruction **malloc** retourne une adresse mémoire valide (valeur non nulle). Sinon, elle retourne **NULL**.

III.5- Libération (destruction)

- ✓ L'instruction **free(L) ;** permet de détruire (supprimer) un élément créé avec l'instruction **malloc** et pointé par le pointeur **L**.

IV- Exercice

Objectif : Maîtriser les listes à travers la recherche des nombres premiers.

Dans ce TP, on s'intéresse à une liste d'entiers, mais les principes abordés restent valables pour des listes de n'importe quel type de données.

Le but de cet exercice est de trouver tous les nombres premiers inférieurs à un entier **Max** (par exemple **10⁶**).

Une méthode rapide pour vérifier si un nombre entier **N** est premier est de tester s'il accepte un diviseur parmi les nombres premiers (déjà connus) qui sont inférieurs à sa racine carrée.

Nous utilisons une liste linéaire chaînée pour stocker les nombres premiers. A chaque fois qu'un nombre premier est découvert (trouvé), il est inséré à la fin de cette liste.

Q1) Écrire une procédure **InsererTete(L, N)** qui permet d'insérer un entier **N** au début de la liste **L** (La liste **L** peut être vide).

Q2) Écrire une procédure **AfficherListe(L)** qui permet d'afficher les éléments d'une liste **L**. Si **L** est vide, la procédure doit afficher le message **'* Liste vide *'**.

Q3) Tester les deux procédures écrites en Q1 et Q2 : Dans le programme principal (fonction **int main()**), créer une liste vide (**L**) ; Insérer dans **L**, les nombres suivants : 2, 3, 5, 7 puis afficher le contenu de cette liste. Dans quel ordre les nombres 2, 3, 5 et 7 sont affichés ?

Q4) Écrire une procédure **InsererQueue(L, N)** qui permet d'insérer un entier **N** à la fin de la liste **L** (**L** peut être vide). **Remarque** : Il est possible d'utiliser la procédure **InsererTete**, si nécessaire.

Q5) Dans le programme principal, remplacer les insertions en tête de la liste par des insertions à la fin. Cette fois-ci, dans quel ordre les nombres 2, 3, 5 et 7 sont affichés ?

Q6) Écrire une fonction **Premier(L, N)** où **L** est une liste qui contient les nombres premiers déjà découverts et **N** un entier. Cette fonction teste si **N** est un nombre premier ou non en vérifiant si **N** accepte un diviseur parmi les nombres premiers qui sont déjà dans la liste **L** et sont inférieurs à sa racine carrée.

Q7) Écrire une procédure **Lister_Nombres_Premiers(L, Max)** qui permet la construction de la liste **L** en cherchant tous les nombres premiers compris entre 2 et **Max**.

Le principe de cette procédure est le suivant :

- On ne teste que les nombres impairs, car tous les nombres pairs sont divisibles par 2.
- Au début la liste L est vide ($L = \{ \}$). - On rajoute 3 à la liste ($L = \{ 3 \}$).
- On commence la recherche par le nombre 5, le premier nombre dans la liste est 3, mais 3 est supérieur à la racine carrée de 5 alors 5 est premier et est ajouté à la liste ($L = \{ 3, 5 \}$).
- On passe à 7, le premier nombre dans la liste est 3, mais 3 est supérieur à la racine carrée de 7 alors 7 est premier et est ajouté à la liste ($L = \{ 3, 5, 7 \}$).
- On passe à 9, on le divise par 3, 3 est un diviseur de 9, alors 9 n'est pas premier.
- On passe à 11, on le divise par 3, il n'est pas divisible par 3. Le nombre premier suivant dans la liste est 5, mais 5 est supérieur à la racine carrée de 11 alors 11 est premier et est ajouté à la liste ($L = \{ 3, 5, 7, 11 \}$).
- On passe à 13, 15, 17, 19, ... et on continue, avec le même principe, jusqu'à trouver **tous les nombres premiers impairs** inférieurs ou égaux à **Max**.
- A la fin, on rajoute au début de la liste, le nombre 2 qui est le **seul nombre premier pair**.

Q8) Écrire une fonction **Longueur(L)** qui donne le nombre d'éléments de la liste L .

Combien de nombre premier inférieur à 10^6 ?

Q9) Écrire une fonction **Somme(L)** qui donne la somme des éléments de la liste L (La somme de tous les nombres premiers inférieurs ou égaux à **Max**).

Q10) Écrire une procédure **SupprimerValeur(L, N)** qui permet de supprimer la valeur N de la liste L .

Q11) Un programme bien conçu, doit impérativement supprimer tous les éléments créés dynamiquement (avec l'instruction **malloc**) en utilisant l'instruction **free**. Écrire une procédure **SupprimerListe(L)** qui permet de supprimer tous les éléments de la liste L . Appeler cette procédure à la fin du programme principal.

Q12) Donner une version récursive des fonctions/procédures suivantes : **AfficherListe**, **InsererQueue**, **Longueur**, **Somme**, **SupprimerValeur**, **SupprimerListe**.